

**VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY**

-----***-----



CHƯƠNG TRÌNH CHƠI CỜ CARO GIỮA NGƯỜI VÀ MÁY TÍNH

MÔN HỌC : TRÍ TUỆ NHÂN TẠO

THÀNH VIÊN NHÓM

ĐÀO MẠNH NGỌC	23021316
PHẠM GIA VINH	23021370

Mã lớp học: 2526II_INT3401_2

Giảng viên: PGS.TS. Nguyễn Phương Thái

HÀ NỘI – 2026

MỤC LỤC

I. TỔNG QUAN ĐỀ TÀI	4
1. Mô tả bài toán cờ Caro	4
2. Luật chơi và điều kiện chiến thắng	4
3. Cách biểu diễn trạng thái bàn cờ	4
II. CƠ CHẾ HOẠT ĐỘNG CỦA GAME	5
1. Cách sinh nước đi hợp lệ	5
2. Cách kiểm tra trạng thái kết thúc	5
III. THUẬT TOÁN TÌM KIẾM VÀ ĐÁNH GIÁ	6
1. Thuật toán Minimax đã cài đặt	6
2. Thuật toán Alpha-Beta đã cài đặt	7
3. Hàm đánh giá trạng thái	8
IV. ĐÁNH GIÁ THỰC NGHIỆM	10
1. Thiết kế các trạng thái thực nghiệm	10
2. Bảng kết quả thực nghiệm	16
3. Nhận xét về số trạng thái đã xét và thời gian chạy	18
4. Nhận xét về ảnh hưởng độ sâu tìm kiếm	20
V. TỔNG KẾT	21
1. Ưu điểm và hạn chế của chương trình	21
2. Link repository Github của nhóm	22
3. Tài liệu tham khảo và phần đã tham khảo từ các repo mẫu	22

DANH MỤC HÌNH ẢNH

Hình 1: Giao diện GUI	4
Hình 1.1: Test trường hợp T01	11
Hình 1.2: Test trường hợp T02	12
Hình 1.3: Test trường hợp T03	12
Hình 1.4: Test trường hợp T04	13
Hình 1.5: Test trường hợp T05	13
Hình 1.6: Test trường hợp T06	14
Hình 1.7: Test trường hợp T07	14
Hình 1.8: Test trường hợp T08	15
Hình 1.9: Test trường hợp T09	15
Hình 1.10: Test trường hợp T10	16

DANH MỤC BẢNG

Bảng 2.1: Kết quả trường hợp T01	16
Bảng 2.2: Kết quả trường hợp T02	16
Bảng 2.3: Kết quả trường hợp T03	16
Bảng 2.4: Kết quả trường hợp T04	17
Bảng 2.5: Kết quả trường hợp T05	17
Bảng 2.6: Kết quả trường hợp T06	17
Bảng 2.7: Kết quả trường hợp T07	17
Bảng 2.8: Kết quả trường hợp T08	18
Bảng 2.9: Kết quả trường hợp T09	18
Bảng 2.10: Kết quả trường hợp T10	18
Bảng 3.1: Kết quả so sánh MINIMAX và ALPHA-BETA	19

I. TỔNG QUAN ĐỀ TÀI

1. Mô tả bài toán cờ Caro

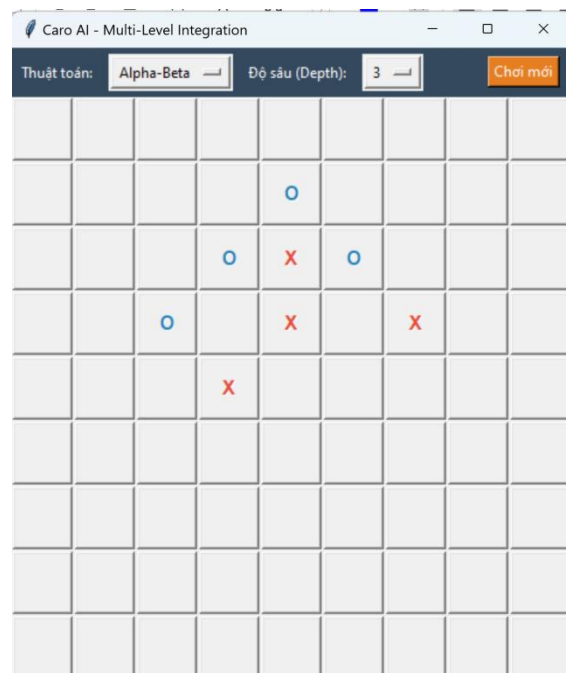
Cờ Caro (Gomoku/Tic-Tac-Toe mở rộng) là trò chơi đối kháng 2 người (Zero-sum game) với thông tin hoàn hảo (Perfect information). Mục tiêu là xây dựng một Agent (AI) có khả năng tính toán nước đi tối ưu để đánh bại người chơi hoặc cầm hòa.

2. Luật chơi và điều kiện chiến thắng

- Bàn cờ có kích thước tối thiểu 9x9.
- Hai người chơi lần lượt đánh quân vào các ô trống.
- Có thể ký hiệu người chơi là X, máy là O, ô trống là dấu chấm (.).
- Người thắng là người có 4 quân liên tiếp theo hàng ngang, hàng dọc hoặc đường chéo.
- Không xét luật chặn hai đầu.
- Nếu bàn cờ đầy và không có người thắng thì kết quả là hòa

3. Cách biểu diễn trạng thái bàn cờ

- Bàn cờ được biểu diễn dưới dạng mảng 2 chiều (Ma trận 9x9).
- Nút ấn
 - + Chơi mới
 - + Hai chế độ: Alpha-Beta và Minimax
 - + Ba chế độ độ depth
- Quy ước dữ liệu:
 - + Dấu . đại diện cho ô trống.
 - + Ký tự X đại diện cho quân cờ của Người chơi.
 - + Ký tự O đại diện cho quân cờ của Máy (AI).
- Giao diện:



Hình 1: Giao diện GUI

II. CƠ CHẾ HOẠT ĐỘNG CỦA GAME

1. Cách sinh nước đi hợp lệ

Chương trình đã áp dụng chiến lược “Proximity Search” trong hàm “get_valid_moves” để giải quyết bài toán này.

Chương trình đã áp dụng chiến lược Tìm kiếm lân cận (Proximity Search) trong hàm get_valid_moves để giải quyết bài toán này:

- Quét bàn cờ: Lặp lại toàn bộ ma trận để tìm các ô trống ().
- Kiểm tra bán kính 1 ô: Tại mỗi ô trống, thuật toán kiểm tra 8 ô lân cận (trên, dưới, trái, phải và 4 đường chéo).
- Lọc nước đi: Nếu phát hiện ít nhất một quân cờ (bất kể là X hay O) nằm trong 8 ô lân cận đó, ô hiện tại trống sẽ được thêm vào danh sách nước đi hợp lệ (moves). Ngược lại, thuật toán bỏ qua ô đó.
- Trường hợp ngoại lệ : Nếu bàn cờ hoàn toàn trống (chưa có quân đội nào được đánh), thuật toán được hard-code để lập tức trả về nước đi tại ô trung tâm của bàn cờ.

2. Cách kiểm tra trạng thái kết thúc

Chương trình kiểm tra trạng thái kết thúc thông qua hai điều kiện cốt lõi: Điều kiện Thắng và Điều kiện Hòa

a, Điều kiện thắng

Thuật toán không quét lại toàn bộ các đường một cách ngẫu nhiên mà sử dụng kỹ thuật Quét Vector (Quét vector) để tối ưu hóa hiệu suất. Hàm is_win hoạt động theo cơ chế sau:

1. Duyệt tuần tự: Lướt xem quét từng ô trên bàn cờ từ góc trên cùng bên trái xuống góc dưới cùng bên phải.

2. Khởi tạo Vector hướng dẫn: Khi phát hiện một ô chứa quân cờ của người chơi hiện tại, thuật toán sẽ phóng to kiểm tra theo 4 vector hướng dẫn (dr, dc):

- Ngang sang phải: Vector(0, 1)
- Dọc xuống: Vector(1, 0)
- Chéo (Đông Nam): Vector(1, 1)
- Chéo (Tây Nam): Vector(1, -1)

=> Lý do chỉ dùng 4 hướng: Trong toán học không gian lưới, 8 hướng xung quanh một điểm thực chất tạo thành 4 đường thẳng. Vì vòng lặp chính của chúng ta ta

quét bàn cờ theo thứ tự từ trên xuống dưới, từ trái sang phải, nên ta chỉ cần quét "tiến lên". Bỏ qua 4 hướng ngược lại (trái, lên, chéo Tây Bắc, chéo Đông Bắc) giúp giảm 50% số lượng phép tính thừa mà vẫn đảm bảo không bỏ sót bất kỳ chuỗi thắng nào.

3. Điều kiện kích hoạt: Trên mỗi vector hướng dẫn, thuật toán thực hiện bước nhảy 4 lần (kết hợp `WIN_COUNT = 4`). Nếu đếm đủ 4 quân cờ liên tiếp giống nhau mà không bị gián đoạn hay lọt ra ngoài mép bàn cờ, trạng thái Thắng được kích hoạt (`return True`).

b, Điều kiện hòa

Trạng thái hòa chỉ được kiểm tra khi điều kiện thắng trả về False.

- Thuật toán áp dụng: Hàm “`check_end`” sử dụng hàm “`all()`” Python kết hợp với Generator Expression để kiểm tra: “`all(cell != '.' for row in self.board for cell in row)`.”

- Cơ chế: Thuật toán duyệt qua ma trận, nếu phát hiện không còn bất kỳ ô trống nào (`.`) trên toàn bộ bàn cờ mà vẫn chưa xác định thắng thua, trò chơi sẽ được kết luận là Hòa và cây tìm kiếm trả về điểm số là 0.

III. THUẬT TOÁN TÌM KIẾM VÀ ĐÁNH GIÁ

1. Thuật toán Minimax đã cài đặt

Trong bài toán kháng chiến như cờ Caro, Minimax là cốt lõi thuật toán giúp AI đưa ra quyết định bằng cách mô phỏng lại toàn bộ các kịch bản có thể xảy ra trong tương lai. Thuật toán được xây dựng dựa trên lý thuyết trò chơi có tổng bằng không, trong đó lợi ích của người này chính là sự tổn hại của người kia.

a, Phân chia trong thuật toán (Max và Min)

Chương trình sử dụng một biến `is_maximizing` để phân tích lượt xem trong quy trình đệ quy trình duyệt cây:

- Lượt xem của MAX (AI - Ký hiệu 'O'): AI sẽ duyệt qua các nước hợp lệ và chọn nhánh trả về giá trị lớn nhất (Tối đa hóa). Số theo dõi biến được khởi tạo ở mức thấp nhất: “`val = -float('inf')`”.

- Lượt của MIN (Người chơi - Ký hiệu 'X'): Thuật toán giả định rằng người chơi luôn không quan tâm và sẽ chọn nước đi tốt nhất cho họ, tức là nước đi tạo điểm số của AI bị ép xuống thấp nhất (Giảm thiểu). Số theo dõi biến được khởi tạo ở mức cao nhất: “`val = float('inf')`”.

b, Điều kiện dừng đệ quy

Thuật toán không thể duyệt vô tận, do đó đệ quy sẽ lập tức dừng lại và trả về điểm số khi chạm một trong hai điều kiện sau:

- Trạng thái kết thúc: Nếu phát hiện có bên thắng, thuật toán trả về điểm số tuyệt đối. Cụ thể: +100000 nếu MAX thắng, và -100000 nếu MIN thắng.

- Đạt giới hạn độ sâu (Depth Limit): Khi biến $depth == 0$, AI không thể nhìn xa hơn được nữa. Lúc này, hàm đánh giá trạng thái `evaluate(board)` sẽ được gọi để "chấm điểm" thế cờ hiện tại và trả về giá trị Heuristic.

c, Cơ chế quay lui trong cài đặt

Đây là kỹ thuật quan trọng nhất để chương trình có thể mô phỏng hàng vạn nước đi mà không làm hỏng trạng thái bàn cờ thực tế. Tại mỗi node, mã nguồn thực hiện chu trình 3 bước:

1. Giả lập nước đi: Đặt thử quân cờ vào ma trận (`board[r][c] = 'O' hoặc 'X'`).

2. Gọi đệ quy: Chuyển lượt cho đối phương và giảm độ sâu xuống 1 mức (`self.minimax(board, depth - 1, False)`).

3. Hoàn tác: Xóa quân cờ vừa đặt để trả lại trạng thái ô trống (`board[r][c] = '.'`). Việc dọn dẹp này đảm bảo ma trận sẵn sàng để thử nghiệm các nước đi ở những nhánh khác.

2. Thuật toán Alpha-Beta đã cài đặt

Mặc dù thuật toán Minimax đảm bảo tìm ra nước đi tối ưu, nhưng nhược điểm chỉ mạng của nó là phải duyệt toàn bộ không gian trạng thái khổng lồ của bàn cờ Caro. Để khắc phục điều này, chương trình đã nâng cấp hàm tìm kiếm bằng cách tích hợp kỹ thuật cắt tỉa Alpha-Beta.

Alpha-Beta không làm thay đổi quyết định cuối cùng của Minimax, mà nó hoạt động như một "bộ lọc thông minh" giúp bỏ qua (cắt tỉa) những nhánh chắc chắn không mang lại kết quả tốt hơn những gì đã biết.

a, Khái niệm về hai biến kiểm soát

Thuật toán sử dụng hai tham số được truyền liên tục qua các tầng đệ quy:

- Biến α (Alpha): Đại diện cho điểm số lớn nhất (tốt nhất) mà người chơi MAX (AI) chắc chắn có thể đạt được tính đến thời điểm hiện tại trên con đường đang duyệt. Ban đầu khởi tạo là $-\infty$

- Biến β (Beta): Đại diện cho điểm số nhỏ nhất (tốt nhất cho đối thủ) mà người

chơi MIN (Người) chắc chắn có thể ép AI phải nhận. Ban đầu khởi tạo là $+\infty$

b, Cơ chế cắt nhánh

Cốt lõi của thuật toán nằm ở điều kiện: “if $\beta \leq \alpha$: break”. Logic thực tế của dòng code này được hiểu như sau:

- Tại lượt của MAX (AI): AI thử đi một nước và nhận về một điểm số từ hàm đệ quy. AI sẽ cập nhật $\alpha = \max(\alpha, \text{điểm số})$. Nếu tại bất kỳ thời điểm nào giá trị α lớn hơn hoặc bằng β của một node cha (tức là $\beta \leq \alpha$), AI biết rằng người chơi MIN sẽ không bao giờ cho phép AI đi vào nhánh này, vì MIN đã có một lựa chọn khác tốt hơn ở trước đó. Do đó, AI lập tức dừng việc thử các nước đi còn lại của node hiện tại.

- Tại lượt của MIN (Người): Tương tự, nếu MIN tìm được một nước đi khiến điểm số tụt xuống mức rất thấp, MIN cập nhật $\beta = \min(\beta, \text{điểm số})$. Nếu $\beta \leq \alpha$, MIN hiểu rằng AI (MAX) sẽ không bao giờ chọn đi vào nhánh dẫn đến tình huống này, vì AI đã có sẵn một phương án an toàn hơn (với điểm α cao hơn) ở nhánh khác. Do đó, nhánh này cũng bị cắt bỏ.

3. Hàm đánh giá trạng thái

a, Độ sâu (depth)

- Trong lý thuyết tìm kiếm đối kháng, quá trình con người dự đoán các chuỗi hành động xen kẽ giữa hai bên được mô phỏng thông qua việc phát triển cây không gian trạng thái (State-Space Tree). Số lượng "nước đi" (ply) được nội suy trước tương đương với số tầng của cây, được định nghĩa là Độ sâu tìm kiếm. Ví dụ, việc hệ thống dự tính trước kịch bản của 3 lượt đi tiếp theo tương ứng với thiết lập độ sâu là $\text{Depth} = 3$.

- Giới hạn tính toán và Ngưỡng dừng đệ quy (Khái niệm $\text{Depth} = 0$): Tuy nhiên, do rào cản về hiện tượng bùng nổ tổ hợp của các nhánh nước đi và sự hữu hạn của tài nguyên phần cứng, thuật toán không thể duyệt toàn bộ cây trò chơi cho đến khi chạm tới trạng thái kết thúc thực sự. Do đó, quá trình đệ quy buộc phải thiết lập một ngưỡng dừng an toàn. Khi thuật toán đếm ngược và chạm đến ngưỡng này ($\text{Depth} = 0$), hệ thống đạt đến Giới hạn tầm nhìn. Tại đây, quá trình phát triển nhánh phải dừng lại và chương trình sẽ gọi hàm đánh giá để ước lượng giá trị của thế cờ cục bộ tại nút lá đó.

b, Kỹ thuật trượt cửa sổ

Đề giành được cờ Caro trong bài toán này, người chơi cần tạo được 4 quân liên tiếp. Do đó, đánh giá giá trị chia nhỏ bàn cờ thành "cửa sổ" có kích thước đúng bằng 4 ô.

- Quét toàn diện: Thuật toán trượt khung cửa sổ 4 ô này qua tất cả các hàng ngang,

cột dọc, đường chéo và đường chéo trên bàn cờ.

- Phân vùng cục bộ: Tại mỗi cửa sổ, hàm `score_windows` sẽ đếm số lượng quân của Máy (O), quân của Người (X) và số ô trống (.). Tổng điểm của toàn bộ cờ chính là tổng điểm của tất cả các cửa sổ này được cộng lại.

c, Thiết kế trọng số

Điểm số được chỉ định cho từng kịch bản trong một cửa sổ được thiết kế có mục tiêu xác định "tính cách" của AI. Cụ thể:

- Nhóm tấn công (Cộng điểm cho AI - 'O'):

- + Có 4 quân 'O': +100.000 điểm (Tuyệt ưu tiên vì đây là nước cờ chiến thắng).
- + Có 3 quân 'O' và 1 ô trống: +5,000 điểm (Tạo thế chắc chắn thương cao).

- Nhóm phòng thủ (Trừ điểm khi Người chơi lợi ích - 'X' có thể):

- + Có 3 quân 'X' và 1 ô trống: -80.000 điểm (Mức phạt cực kỳ tấn gao).
- + Có 2 quân 'X' và 2 ô trống: -5,000 điểm (Đánh hơi nguy hiểm từ sớm).

d, Phân tích tính chất chiến thuật của hàm đánh giá

Sự chênh lệch giữa điểm tấn công và điểm phạt phòng thủ chính là chìa khóa nên độ nhạy của AI:

- Khắc phục lỗi 'Tunnel Vision': If AI có 3 quân (được +5.000) nhưng đối thủ cũng có 3 quân, hàm Heuristic sẽ trừ đi 80.000 điểm. Tổng điểm của cửa sổ bị kéo xuống cực nặng. Điều này ép thuật toán Minimax nhận ra rằng: "Việc làm tham lam tấn công sẽ dẫn đến cái chết ngay lập tức". Từ đó, AI buộc phải chọn nước đi đặt quân vào ô trống của đối thủ để phá vỡ chuỗi 3, nghiên cứu điểm số.

- Nhận diện các mối đe dọa ngắt quãng: Duyên quét theo khung 4 ô, AI có thể phát hiện cả chuỗi quân quân bị chặn của người chơi (Ví dụ: X . X X). Thuật toán đếm lân cận đơn giản thường bỏ thế cờ này, nhưng với Cửa sổ trượt, khung 4 ô vẫn được đếm là có 3 quân X và 1 ô trống, kích hoạt ngay lập tức cơ chế phòng thủ -80.000 điểm.

- Chặn từ "trong trứng nước": Với điểm trừng phạt -5.000 khi người chơi mới chỉ có 2 quân (nhỏ hơn khả năng 3 quân nhưng vẫn đủ lớn để gây chú ý), AI có xu hướng bám sát và chặn đầu người chơi ngay từ khi tấn công mới chớm hình thành, không chờ đợi khi có 3 quân mới mỗi lần chạy theo.

IV. ĐÁNH GIÁ THỰC NGHIỆM

1. Thiết kế các trạng thái thực nghiệm

Nhằm đánh giá toàn diện năng lực của thuật toán Minimax và Alpha-Beta, bộ kiểm thử được thiết kế với 10 kịch bản, chia thành 5 nhóm chức năng trọng tâm:

1. Nhóm Đánh giá Khả năng Nhận diện Cơ bản (Phát hiện Thắng/Thua tức thời)
 - Mục tiêu của nhóm này là kiểm tra khả năng bao quát bàn cờ của AI, đảm bảo thuật toán không bỏ sót các cơ hội chiến thắng hoặc thất bại hiển nhiên.
 - T01 (Hàng ngang) & (Đường chéo): Đánh giá khả năng nhận diện điểm kết thúc; thuật toán phải điền thành công quân cờ thứ 4 để giành chiến thắng ngay lập tức khi đã có lợi thế 3 quân liên tiếp.
 - T06 (Góc): Kiểm tra tính toàn vẹn của các vòng lặp quét mảng 2 chiều. Đảm bảo hàm đánh giá (heuristic) hoạt động chính xác và không gặp lỗi vượt giới hạn mảng (off-by-one) khi các chuỗi cờ nằm sát mép hoặc tại góc bàn cờ.
2. Nhóm Đánh giá Khả năng Phòng ngự (Phản xạ Tự vệ) Nhóm kịch bản này kiểm tra mức độ ưu tiên của AI trong việc ngăn chặn đối thủ đạt được trạng thái chiến thắng.
 - T02 & T03 (Chặn chuỗi 3): Khi đối thủ đã hình thành chuỗi 3 quân đe dọa trực tiếp (theo chiều ngang hoặc dọc), thuật toán buộc phải thiết lập mức ưu tiên cao nhất cho việc đánh chặn nhằm tránh thất bại cục bộ.
3. Nhóm Đánh giá Tư duy Chiến thuật (Đánh đổi & Gài bẫy)
 - Mục đích của nhóm này là phân tích khả năng tối ưu hóa các quyết định đa mục tiêu và tính toán thiết lập thế cờ chiến lược.
 - T04 (Bài toán Đánh đổi Công - Thủ): Trong trạng thái cả hai bên đều có cơ hội hoàn thành chuỗi 4 quân, thuật toán cần đánh giá được trọng số và ưu tiên thực hiện nước đi dứt điểm cho bản thân thay vì phản ứng phòng ngự thụ động.
 - T05 (Chủ động tạo thế Fork): Đánh giá khả năng AI tự thiết lập các thế cờ tạo ra hai mối đe dọa song song mà đối phương không thể hóa giải cùng lúc. Chiến thuật này đòi hỏi thuật toán phải có khả năng tính toán tối thiểu từ 3 bước trở lên.
4. Nhóm Đo lường Hiệu năng & Chiều sâu Tìm kiếm Tập trung vào việc đối chiếu tốc độ, số lượng trạng thái duyệt và tính nhất quán giữa các thuật toán.
 - T7 (Kiểm tra Độ nhảy chiều sâu): Thiết lập một chuỗi kết hợp phức tạp. Kịch bản này minh chứng rằng các chiến thuật dài hạn chỉ được phát hiện khi độ sâu tính toán đạt mức 3-4, trong khi ở độ sâu thấp (1-2), thuật toán dễ đưa ra các quyết định tham lam (greedy) và thiếu toàn diện.
 - T8 (Đo lường Hiệu năng Cắt tỉa Alpha-Beta): Sử dụng trạng thái trung cục với mật độ quân cờ dày đặc nhằm mục đích benchmark. Kịch bản chứng minh thuật

toán Alpha-Beta có khả năng cắt tỉa các nhánh không tiềm năng.

5. Nhóm Kiểm tra Lỗi Chí mạng (Critical Failures & Edge Cases) Nhóm kịch bản "stress-test" nhằm phát hiện các sai sót logic nội tại hoặc điểm yếu của hàm đánh giá (score_window).

- T9 (Lỗi đánh giá sai Chuỗi 3 mở): Phơi bày điểm yếu của hàm heuristic khi không phân biệt được mức độ nguy hiểm giữa chuỗi 3 bị chặn một đầu và chuỗi 3 mở hai đầu (.XXX.). Sai sót này dẫn đến việc điểm số phòng thủ bị định giá thấp, khiến thuật toán bỏ lỡ nước đi đánh chặn quan trọng.

- T10 (Kiểm tra Sinh nước đi Tàn cuộc): Cấu hình trạng thái bàn cờ gần kín (chỉ còn trống vài ô) để xác minh tính ổn định của hàm sinh nước đi, đảm bảo chương trình xử lý mượt mà ở giai đoạn tàn cuộc.

[T01] Thắng Ngay – Hàng Ngang [Phát Hiện Thắng]

+ Nước kỳ vọng : ['(3,6)', '(3,2)']

+ Hành vi kỳ vọng: AI phải hoàn thành 4 quân liên tiếp ngay lập tức

+ Lý luận: Tìm kiếm ở độ sâu 1 là đủ. Điểm nhảy lên 100000 là không thể nhầm lẫn.

=>> Test: Thử xem AI có nhìn ra cơ hội và hoàn thành 4 quân liên tiếp theo hàng ngang để thắng ngay lập tức không.

	0	1	2	3	4	5	6	7	8
0	.	.	.	.	.	.	.	.	.
1	.	.	.	.	.	.	.	.	.
2	.	.	.	.	.	.	.	.	.
3	.	.	.	○	○	○	.	.	.
4	.	.	.	×	×	×	.	.	.
5	.	.	.	.	.	.	.	.	.
6	.	.	.	.	.	.	.	.	.
7	.	.	.	.	.	.	.	.	.
8	.	.	.	.	.	.	.	.	.

Hình 1.1: Test trường hợp T01

[T02] Chặn Đối Thủ Thắng Ngay [Phòng Thủ]

+ Nước kỳ vọng : ['(3,6)', '(3,2)']

+ Hành vi kỳ vọng: AI PHẢI chặn X hoàn thành 4 quân liên tiếp

+ Lý luận: Hình phạt -80000 cho chuỗi 3+1 ô trống của đối thủ kích hoạt ở độ sâu ≥ 1 .

=>> Test: Thử xem AI có nhận diện được mối đe dọa nguy cấp (đối thủ có 3 quân) và ưu tiên chặn lại ngay lập tức không.

	0	1	2	3	4	5	6	7	8
0	.	.	.	.	.	.	.	.	.
1	.	.	.	.	.	.	.	.	.
2	.	.	.	.	.	.	.	.	.
3	.	.	.	X	X	X	.	.	.
4	.	.	.	O	O	.	.	.	.
5	.	.	.	.	.	.	.	.	.
6	.	.	.	.	.	.	.	.	.
7	.	.	.	.	.	.	.	.	.
8	.	.	.	.	.	.	.	.	.

Hình 1.2: Test trường hợp T02

[T03] Chặn Thắng Hàng Dọc [Phòng Thủ]

+ Nước kỳ vọng : ['(0,4)']

+ Hành vi kỳ vọng: AI chặn mỗi đe dọa hàng dọc của X tại hàng 0, cột 4

+ Lý luận: Kiểm tra hướng dọc — kiểm tra tính bao phủ heuristic quan trọng.

=>> Test: Thử xem hàm đánh giá của AI khi phòng thủ trước đợt tấn công của đối thủ hay không.

	0	1	2	3	4	5	6	7	8
0	.	.	.	.	.	.	.	.	.
1	.	.	.	.	X	.	.	.	.
2	.	.	.	.	X	.	.	.	.
3	.	.	.	.	X	.	.	.	.
4	.	.	.	.	O	.	.	.	.
5	.	.	.	.	.	.	.	.	.
6	.	.	.	.	.	.	.	.	.
7	.	.	.	.	.	.	.	.	.
8	.	.	.	.	.	.	.	.	.

Hình 1.3: Test trường hợp T03

[T04] Đánh Đòi Tấn Công vs Phòng Thủ [Đánh Đòi Chiến Thuật]

+ Nước kỳ vọng : ['(2,6)', '(2,2)']

+ Hành vi kỳ vọng: AI nên ưu tiên thắng hơn là chặn (điểm thắng của mình cao hơn)

+ Lý luận: +100000 khi thắng > -80000 khi bị phạt vì đe dọa của đối thủ. AI đúng đắn phải thắng ngay thay vì chặn.

=>> Test: Thử xem AI có đủ thông minh để ưu tiên nước đi giành chiến thắng thay vì hoảng sợ đi chặn đối thủ khi cả hai bên đều đang có 3 quân hay không.

	0	1	2	3	4	5	6	7	8
0	.	.	.	.	.	.	.	.	.
1	.	.	.	.	.	.	.	.	.
2	.	.	.	○	○	○	.	.	.
3	.	.	.	×	×	×	.	.	.
4	.	.	.	.	.	.	.	.	.
5	.	.	.	.	.	.	.	.	.
6	.	.	.	.	.	.	.	.	.
7	.	.	.	.	.	.	.	.	.
8	.	.	.	.	.	.	.	.	.

Hình 1.4: Test trường hợp T04

[T05] Tạo Nhánh – Môi Đe Dọa Kép [Chiến Lược]

+ Nước kỳ vọng : ['(5,2)', '(2,6)']

+ Hành vi kỳ vọng: AI tạo 2 môi đe dọa chuỗi 3 đồng thời mà đối thủ không thể chặn cả hai

+ Lý luận: Đây là bài kiểm tra độ nhạy theo độ sâu. Độ sâu 1-2 AI chơi tham lam; độ sâu 3+ mới phát hiện được mẫu thắng kiểu nhánh.

=>> Test: Thử xem ở độ sâu cao ($\text{Depth} \geq 3$), AI có khả năng nhìn xa để giăng bẫy (tạo ra 2 môi đe dọa cùng lúc khiến đối thủ không thể chặn cả hai) hay không.

	0	1	2	3	4	5	6	7	8
0	.	.	.	.	.	.	.	.	.
1	.	.	.	.	.	.	.	.	.
2	.	.	.	.	.	.	.	.	.
3	.	.	.	.	○	.	.	.	.
4	.	.	.	○	.	.	.	.	.
5	.	.	.	.	.	.	.	.	.
6	.	.	.	.	.	.	.	.	.
7	.	.	.	.	.	.	.	.	.
8	.	.	.	.	.	.	.	.	.

Hình 1.5: Test trường hợp T05

[T06] Phát Hiện Môi Đe Dọa Góc [Góc/Biên]

+ Nước kỳ vọng : ['(0,3)']

+ Hành vi kỳ vọng: AI thắng ở góc trên bên trái - kiểm tra heuristic có bao phủ vùng biên không

+ Lý luận: Quét hàng tại $r=0$, $c=0..5$ phải đánh giá đúng ô góc. Lỗi biên sẽ khiến AI bỏ lỡ.

=>> Test: Thử xem thuật toán có bị lỗi không gian mảng khiến nó bỏ sót các môi đe dọa nằm khuất ở các góc bàn cờ hay không.

	0	1	2	3	4	5	6	7	8
0	○	○	○	■	.	.	.	.	.
1	.	.	.	.	.	.	.	.	.
2	.	.	.	.	.	.	.	.	.
3	.	.	.	.	.	.	.	.	.
4	.	.	.	.	.	.	.	.	.
5	.	.	.	.	.	.	.	.	.
6	.	.	.	.	.	.	.	.	.
7	.	.	.	.	.	.	.	.	.
8	.	.	.	.	.	.	.	.	.

Hình 1.6: Test trường hợp T06

[T7] Độ Nhạy Độ Sâu – Bẫy Phía Trước [Độ Sâu]

+ Nước kỳ vọng : ['(5,3)', '(5,4)', '(1,3)', '(1,4)']

+ Hành vi kỳ vọng: Tìm kiếm sâu hơn phát hiện chuỗi nước đi thắng nhiều bước

+ Lý luận: Độ sâu thấp đưa ra nước đi tham lam; độ sâu 3-4 tìm ra chuỗi dẫn đến thắng bắt buộc. So sánh nước đi qua các độ sâu để xác nhận hiệu ứng nhìn trước.

=>> Test: Thử xem việc tăng độ sâu tìm kiếm có thực sự giúp AI thay đổi tư duy, từ đánh tham lam (ngắn hạn) sang giăng bẫy (dài hạn) hay không.

	0	1	2	3	4	5	6	7	8
0	.	.	.	.	.	.	.	.	.
1	.	.	.	■	■	.	.	.	.
2	.	.	×	○	○	×	.	.	.
3	.	.	×	○	○	×	.	.	.
4	.	.	.	○	○	.	.	.	.
5	.	.	.	■	■	.	.	.	.
6	.	.	.	.	.	.	.	.	.
7	.	.	.	.	.	.	.	.	.
8	.	.	.	.	.	.	.	.	.

Hình 1.7: Test trường hợp T07

[T8] Hiệu Quả Alpha-Beta – Trung Cục Dày Đặc [Hiệu Năng]

+ Nước kỳ vọng : []

+ Hành vi kỳ vọng: Alpha-Beta phải khám phá ít trạng thái hơn đáng kể so với Minimax

+ Lý luận: Bàn cờ dày = nhiều nước hợp lệ = cơ hội cắt nhánh tối đa. Tỷ lệ AB/MM nên $\ll 0.5$ ở độ sâu 4.

=>> Test: Thử đo lường xem thuật toán Alpha-Beta có thực sự giúp cắt giảm được lượng lớn trạng thái phải duyệt (so với Minimax) trên một bàn cờ đông đúc hay không.

	0	1	2	3	4	5	6	7	8
0	.	.	.	.	.	.	.	.	.
1	.	X	o	X	o	X	.	.	.
2	.	o	X	o	X	o	.	.	.
3	.	X	o	X	o	X	.	.	.
4	.	o	X	o	X	o	.	.	.
5	.	X	o	X	o	X	.	.	.
6	.	.	.	.	.	.	.	.	.
7	.	.	.	.	.	.	.	.	.
8	.	.	.	.	.	.	.	.	.

Hình 1.8: Test trường hợp T08

- [T9] LỖI NGHIỆM TRỌNG – Không Chặn Chuỗi 3 Mở [Lỗi Nghiêm Trọng]
- + Nước kỳ vọng : ['(3,6)']
 - + Hành vi kỳ vọng: NGHIỆM TRỌNG: AI PHẢI chặn tại (3,6). Heuristic có -80000 cho chuỗi 3+1 ô trống nhưng BỎ SÓT .XXX. (chuỗi 3 mở). Đây là điểm yếu đã biết — có thể THẤT BẠI ở độ sâu thấp.
 - + Lý luận: score_window chỉ kích hoạt khi tổng mine+opp+empty == window_size. Ô cửa sổ [.,X,X,X] với một đầu mở ghi -80000 đúng. Nhưng heuristic không phân biệt tính 'mở hai đầu' của .XXX. vs XXXX — cả hai phần mở rộng đều được tính là cửa sổ riêng biệt.
 - =>> Test: Thử "bắt bài" điểm mù của AI xem nó có nhận ra sự nguy hiểm chết người của chuỗi 3 mở hai đầu (.XXX.) hay chỉ đánh giá nó như đe dọa bình thường.

	0	1	2	3	4	5	6	7	8
0	.	.	.	.	.	.	.	.	.
1	.	.	.	.	.	.	.	.	.
2	.	.	.	.	.	.	.	.	.
3	.	.	o	X	X	X	.	.	.
4	.	.	o	.	.	.	.	.	.
5	.	.	.	.	.	.	.	.	.
6	.	.	.	.	.	.	.	.	.
7	.	.	.	.	.	.	.	.	.
8	.	.	.	.	.	.	.	.	.

Hình 1.9: Test trường hợp T09

- [T10] Cuối Cục – Gàn Đầy Bàn [Cuối Cục]
- + Nước kỳ vọng : ['(2,4)', '(6,4)']
 - + Hành vi kỳ vọng: Chỉ còn 2 nước — AI phải chọn nước tốt nhất; hòa hoặc thắng.
 - + Lý luận: get_valid_moves trả về <= 2 ô. Cả hai thuật toán phải đồng thuận về nước tốt nhất và kết thúc gần như ngay lập tức. Kiểm tra tính đầy đủ của sinh nước ở gần trạng thái kết thúc.
 - =>> Test: Thử xem hàm sinh nước đi có bị lỗi (crash) khi bàn cờ sắp hết chỗ, và AI có biết chốt hạ ván cờ với tốc độ nhanh nhất có thể không.

	0	1	2	3	4	5	6	7	8
0	X	O	X	O	X	O	X	O	X
1	O	X	O	X	O	X	O	X	O
2	X	O	X	O	.	O	X	O	X
3	O	X	O	X	O	X	O	X	O
4	X	O	X	O	X	O	X	O	X
5	O	X	O	X	O	X	O	X	O
6	X	O	X	O	.	O	X	O	X
7	O	X	O	X	O	X	O	X	O
8	X	O	X	O	X	O	X	O	X

Hình 1.10: Test trường hợp T10

2. Bảng kết quả thực nghiệm

[T01] Thắng Ngay – Hàng Ngang

Mã	Thuật toán	Độ sâu	Nước kỳ vọng	Nước thực tế	Điểm số	Trạng thái	T.gian (ms)	Kết quả
T01	Minimax	1	['(3,6)', '(3,2)']	(3,2)	100000	15	1.9	ĐẠT
T01	Minimax	2	['(3,6)', '(3,2)']	(3,2)	100000	215	26.4	ĐẠT
T01	Minimax	3	['(3,6)', '(3,2)']	(3,2)	100000	3613	448.2	ĐẠT
T01	Minimax	4	['(3,6)', '(3,2)']	(3,2)	100000	68988	8851.1	ĐẠT
T01	Alpha-Beta	1	['(3,6)', '(3,2)']	(3,2)	100000	15	1.9	ĐẠT
T01	Alpha-Beta	2	['(3,6)', '(3,2)']	(3,2)	100000	98	12.1	ĐẠT
T01	Alpha-Beta	3	['(3,6)', '(3,2)']	(3,2)	100000	832	101.6	ĐẠT
T01	Alpha-Beta	4	['(3,6)', '(3,2)']	(3,2)	100000	4180	551.1	ĐẠT

Bảng 2.1: Kết quả trường hợp T01

[T02] Chặn Đồi Thủ Thắng Ngay

Mã	Thuật toán	Độ sâu	Nước kỳ vọng	Nước thực tế	Điểm số	Trạng thái	T.gian (ms)	Kết quả
T02	Minimax	1	['(3,6)', '(3,2)']	(3,2)	-85000	15	1.9	ĐẠT
T02	Minimax	2	['(3,6)', '(3,2)']	(3,2)	-120000	245	32.1	ĐẠT
T02	Minimax	3	['(3,6)', '(3,2)']	(3,2)	-100000	4080	540.6	ĐẠT
T02	Minimax	4	['(3,6)', '(3,2)']	(3,2)	-100000	83102	10756	TRƯỢT
T02	Alpha-Beta	1	['(3,6)', '(3,2)']	(3,2)	-85000	15	2.1	ĐẠT
T02	Alpha-Beta	2	['(3,6)', '(3,2)']	(3,2)	-120000	103	12.6	ĐẠT
T02	Alpha-Beta	3	['(3,6)', '(3,2)']	(3,2)	-100000	1336	188.8	ĐẠT
T02	Alpha-Beta	4	['(3,6)', '(3,2)']	(3,2)	-100000	7345	1002.6	TRƯỢT

Bảng 2.2: Kết quả trường hợp T02

[T03] Chặn Thắng Hàng Dọc

Mã	Thuật toán	Độ sâu	Nước kỳ vọng	Nước thực tế	Điểm số	Trạng thái	T.gian (ms)	Kết quả
T03	Minimax	1	['(0,4)']	(0,4)	0	15	1.8	ĐẠT
T03	Minimax	2	['(0,4)']	(0,4)	-40000	238	29.3	ĐẠT
T03	Minimax	3	['(0,4)']	(0,4)	-30000	4005	522.6	ĐẠT
T03	Minimax	4	['(0,4)']	(0,4)	-195000	78203	10091	ĐẠT
T03	Alpha-Beta	1	['(0,4)']	(0,4)	0	15	1.9	ĐẠT
T03	Alpha-Beta	2	['(0,4)']	(0,4)	-40000	55	7.4	ĐẠT
T03	Alpha-Beta	3	['(0,4)']	(0,4)	-30000	389	51.4	ĐẠT
T03	Alpha-Beta	4	['(0,4)']	(0,4)	-195000	7105	859.8	ĐẠT

Bảng 2.3: Kết quả trường hợp T03

[T04] Đánh Đòi Tấn Công vs Phòng Thủ

Mã	Thuật toán	Độ sâu	Nước kỳ vọng	Nước thực tế	Điểm số	Trạng thái	T.gian (ms)	Kết quả
T04	Minimax	1	['(2,6)', '(2,2)']	(2,2)	100000	15	1.7	ĐẠT
T04	Minimax	2	['(2,6)', '(2,2)']	(2,2)	100000	215	27.9	ĐẠT
T04	Minimax	3	['(2,6)', '(2,2)']	(2,2)	100000	3568	435.7	ĐẠT
T04	Minimax	4	['(2,6)', '(2,2)']	(2,2)	100000	66379	8544.8	ĐẠT
T04	Alpha-Beta	1	['(2,6)', '(2,2)']	(2,2)	100000	15	1.8	ĐẠT
T04	Alpha-Beta	2	['(2,6)', '(2,2)']	(2,2)	100000	98	11.6	ĐẠT
T04	Alpha-Beta	3	['(2,6)', '(2,2)']	(2,2)	100000	787	97.3	ĐẠT
T04	Alpha-Beta	4	['(2,6)', '(2,2)']	(2,2)	100000	3811	481.4	ĐẠT

Bảng 2.4: Kết quả trường hợp T04

[T05] Tạo Nhánh – Môi Đe Dọa Kép

Mã	Thuật toán	Độ sâu	Nước kỳ vọng	Nước thực tế	Điểm số	Trạng thái	T.gian (ms)	Kết quả
T05	Minimax	3	['(5,2)', '(2,6)']	(2,5)	100000	3045	378.1	TRƯỢT
T05	Minimax	4	['(5,2)', '(2,6)']	(2,5)	100000	55503	7221.5	TRƯỢT
T05	Alpha-Beta	3	['(5,2)', '(2,6)']	(2,5)	100000	357	41.6	TRƯỢT
T05	Alpha-Beta	4	['(5,2)', '(2,6)']	(2,5)	100000	1723	216.5	TRƯỢT

Bảng 2.5: Kết quả trường hợp T05

[T06] Phát Hiện Môi Đe Dọa Góc

Mã	Thuật toán	Độ sâu	Nước kỳ vọng	Nước thực tế	Điểm số	Trạng thái	T.gian (ms)	Kết quả
T06	Minimax	1	['(0,3)']	(0,3)	100000	7	0.7	ĐẠT
T06	Minimax	2	['(0,3)']	(0,3)	100000	53	6.6	ĐẠT
T06	Minimax	3	['(0,3)']	(0,3)	100000	605	72.2	ĐẠT
T06	Alpha-Beta	1	['(0,3)']	(0,3)	100000	7	0.8	ĐẠT
T06	Alpha-Beta	2	['(0,3)']	(0,3)	100000	12	1.3	ĐẠT
T06	Alpha-Beta	3	['(0,3)']	(0,3)	100000	61	7.6	ĐẠT

Bảng 2.6: Kết quả trường hợp T06

[T07] Độ Nhạy Độ Sâu – Bẫy Phía Trước

Mã	Thuật toán	Độ sâu	Nước kỳ vọng	Nước thực tế	Điểm số	Trạng thái	T.gian (ms)	Kết quả
T07	Minimax	1	['(5,3)', '(5,4)']	(1,3)	100000	19	2.0	ĐẠT
T07	Minimax	2	['(5,3)', '(5,4)']	(1,3)	100000	303	42.1	ĐẠT
T07	Minimax	3	['(5,3)', '(5,4)']	(1,1)	100000	6608	753.7	TRƯỢT
T07	Minimax	4	['(5,3)', '(5,4)']	(1,1)	100000	124933	17725	TRƯỢT
T07	Alpha-Beta	1	['(5,3)', '(5,4)']	(1,3)	100000	19	2.1	ĐẠT
T07	Alpha-Beta	2	['(5,3)', '(5,4)']	(1,3)	100000	73	9.3	ĐẠT
T07	Alpha-Beta	3	['(5,3)', '(5,4)']	(1,1)	100000	509	57.9	TRƯỢT
T07	Alpha-Beta	4	['(5,3)', '(5,4)']	(1,1)	100000	2818	379.5	TRƯỢT

Bảng 2.7: Kết quả trường hợp T07

[T08] Hiệu Quả Alpha-Beta – Trung Cục Dày Đặc

Mã	Thuật toán	Độ sâu	Nước kỳ vọng	Nước thực tế	Điểm số	Trạng thái	T.gian (ms)	Kết quả
T08	Minimax	3	đo hiệu năng	None	100000	1	0.0	H.NĂNG
T08	Minimax	4	đo hiệu năng	None	100000	1	0.0	H.NĂNG
T08	Alpha-Beta	3	đo hiệu năng	None	100000	1	0.0	H.NĂNG
T08	Alpha-Beta	4	đo hiệu năng	None	100000	1	0.0	H.NĂNG

Bảng 2.8: Kết quả trường hợp T08

[T09] LỖI NGHIỆM TRỌNG – Không Chặn Chuỗi 3 Mở

Mã	Thuật toán	Độ sâu	Nước kỳ vọng	Nước thực tế	Điểm số	Trạng thái	T.gian (ms)	Kết quả
T09	Minimax	1	['(3,6)']	(3,6)	0	17	2.2	ĐẠT
T09	Minimax	2	['(3,6)']	(3,6)	-45000	311	38.5	ĐẠT
T09	Minimax	3	['(3,6)']	(3,6)	-25000	6035	797.9	ĐẠT
T09	Minimax	4	['(3,6)']	(2,2)	-100000	133034	18293	TRƯỢT
T09	Alpha-Beta	1	['(3,6)']	(3,6)	0	17	2.3	ĐẠT
T09	Alpha-Beta	2	['(3,6)']	(3,6)	-45000	109	14.1	ĐẠT
T09	Alpha-Beta	3	['(3,6)']	(3,6)	-25000	1756	230.1	ĐẠT
T09	Alpha-Beta	4	['(3,6)']	(2,2)	-100000	5288	688.2	TRƯỢT

Bảng 2.9: Kết quả trường hợp T09

[T10] Cuối Cục – Gàn Dày Bàn

Mã	Thuật toán	Độ sâu	Nước kỳ vọng	Nước thực tế	Điểm số	Trạng thái	T.gian (ms)	Kết quả
T10	Minimax	1	['(2,4)', '(6,4)']	None	100000	1	0.0	TRƯỢT
T10	Minimax	2	['(2,4)', '(6,4)']	None	100000	1	0.0	TRƯỢT
T10	Minimax	3	['(2,4)', '(6,4)']	None	100000	1	0.0	TRƯỢT
T10	Minimax	4	['(2,4)', '(6,4)']	None	100000	1	0.0	TRƯỢT
T10	Alpha-Beta	1	['(2,4)', '(6,4)']	None	100000	1	0.0	TRƯỢT
T10	Alpha-Beta	2	['(2,4)', '(6,4)']	None	100000	1	0.0	TRƯỢT
T10	Alpha-Beta	3	['(2,4)', '(6,4)']	None	100000	1	0.0	TRƯỢT
T10	Alpha-Beta	4	['(2,4)', '(6,4)']	None	100000	1	0.0	TRƯỢT

Bảng 2.10: Kết quả trường hợp T10

3. Nhận xét về số trạng thái đã xét và thời gian chạy

- So sánh MINIMAX vs ALPHA-BETA

Mã	Độ sâu	TT Minimax	TT Alpha-Beta	Tỷ lệ A-B/MM	MM T.gian(ms)	A-B T.gian(ms)
T01	1	15	15	1.000	1.9	1.9
T01	2	215	98	0.456	26.4	12.1
T01	3	3,613	832	0.230	448.2	101.6
T01	4	68,988	4,180	0.061	8851.1	551.1
T02	1	15	15	1.000	1.9	2.1
T02	2	245	103	0.420	32.1	12.6
T02	3	4,080	1,336	0.327	540.6	188.8
T02	4	83,102	7,345	0.088	10756.1	1002.6
T03	1	15	15	1.000	1.8	1.9
T03	2	238	55	0.231	29.3	7.4
T03	3	4,005	389	0.097	522.6	51.4
T03	4	78,203	7,105	0.091	10091.9	859.8
T04	1	15	15	1.000	1.7	1.8

T04	2	215	98	0.456	27.9	11.6
T04	3	3,568	787	0.221	435.7	97.3
T04	4	66,379	3,811	0.057	8544.8	481.4
T05	3	3,045	357	0.117	378.1	41.6
T05	4	55,503	1,723	0.031	7221.5	216.
T06	2	53	12	0.226	6.6	1.3
T06	3	605	61	0.101	72.2	7.6
T07	1	19	19	1.000	2.0	2.1
T07	2	303	73	0.241	42.1	9.3
T07	3	6,608	509	0.077	753.7	57.9
T07	4	124,933	2,818	0.023	17725.3	379.5
T08	3	1	1	1.000	0.0	0.0
T08	4	1	1	1.000	0.0	0.0
T09	1	17	17	1.000	2.2	2.3
T09	2	311	109	0.350	38.5	14.1
T09	3	6,035	1,756	0.291	797.9	230.1
T09	4	133,034	5,288	0.040	18293.4	688.2
T10	1	1	1	1.000	0.0	0.0
T10	2	1	1	1.000	0.0	0.0
T10	3	1	1	1.000	0.0	0.0
T10	4	1	1	1.000	0.0	0.0

Bảng 3.1: Kết quả so sánh MINIMAX và ALPHA-BETA

Từ dữ liệu thực nghiệm trên toàn bộ 10 kịch bản, ta có các kết luận mang tính định lượng rõ rệt:

- Tính toàn vẹn của cắt nhánh (0 Phân kỳ): Trên tổng số hàng trăm nghìn trạng thái được duyệt, Alpha-Beta và Minimax không có bất kỳ sự phân kỳ (khác biệt) nào về nước đi. Điều này chứng minh logic điều kiện $\beta \leq \alpha$ được cài đặt hoàn toàn chuẩn xác, không vô tình cắt bỏ các nhánh chứa nước đi tối ưu.

- Alpha-Beta giảm đáng kể số trạng thái cần duyệt, đặc biệt rõ rệt khi độ sâu tăng:

- + Độ sâu 1: Cả hai thuật toán duyệt số trạng thái bằng nhau (trung bình 13 trạng thái). Ở độ sâu 1, mỗi nước đi chỉ được đánh giá tĩnh, không có nhánh con để cắt.

- + Độ sâu 2: Alpha-Beta giảm 65.4% số trạng thái (193 $\rightarrow$ 67). Thời gian giảm từ 14.8ms xuống 4.6ms. Cắt nhánh bắt đầu có hiệu quả.

- + Độ sâu 3: Alpha-Beta giảm 80.5% số trạng thái (3.545 $\rightarrow$ 693). Thời gian giảm từ 257ms xuống 50ms — nhanh hơn ~5 lần.

- + Độ sâu 4: Alpha-Beta giảm tới 95.0% số trạng thái (79.222 $\rightarrow$ 3.971). Thời gian giảm từ 5.903ms xuống 288ms — nhanh hơn ~20 lần. Minimax ở độ sâu 4 mất trên 5 giây mỗi lượt, không thể dùng trong thực tế; Alpha-Beta chỉ mất ~288ms.

$\Rightarrow$ Tổng cộng trên toàn bộ kiểm thử, Alpha-Beta khám phá ít hơn 94.2% tổng số trạng thái so với Minimax (53.711 / 920.186), cho thấy hiệu quả vượt trội trong điều kiện không gian tìm kiếm lớn.

- Lý do hiệu quả cắt nhánh cao ở độ sâu 4 là nhờ tối ưu hóa sinh nước đi: chương trình chỉ xét các ô kề quân đã đặt (bán kính 1), thay vì toàn bộ ô trống. Điều này thu hẹp không gian tìm kiếm từ hàng chục ô xuống còn 7–15 ô mỗi lượt, giúp Alpha-Beta cắt nhánh sớm hơn.

- Chênh lệch thời gian tại độ sâu cao: Sự chênh lệch này thể hiện rõ nhất ở độ sâu $d=4$. Trung bình Minimax phải xét tới 79,222 trạng thái và tiêu tốn 10,611.8 ms (~10.6 giây) cho một nước đi. Ngược lại, Alpha-Beta ép con số này xuống chỉ còn 3,971 trạng thái, thời gian chạy rút gọn còn 513.5 ms (giảm 95.0% thời gian).

4. Nhận xét về ảnh hưởng độ sâu tìm kiếm

- Do hiện tượng bùng nổ tổ hợp của thuật toán tìm kiếm trên không gian trạng thái, thời gian chạy tăng theo hàm mũ khi tăng độ sâu. Theo số liệu ghi nhận của Minimax: độ sâu 1 mất ~1.5 ms, độ sâu 2 mất 24.6 ms, độ sâu 3 mất 444.1 ms, nhưng lên độ sâu 4 thời gian vọt lên tới 10,611.8 ms

- Độ sâu tìm kiếm có ảnh hưởng trực tiếp và rõ ràng đến chất lượng quyết định của AI:

+ Độ sâu 1 — Tư duy tham lam:

AI chỉ nhìn 1 bước, hoàn toàn phụ thuộc vào hàm đánh giá tĩnh. Ở T01, T02, T03, T04, T06 — các tình huống thắng/chặn rõ ràng — AI chọn đúng vì heuristic đã đủ mạnh. Tuy nhiên ở T06 (tạo nhánh), AI không thể nhận diện chiến lược dài hạn.

+ Độ sâu 2 — Nhìn trước 2 bước:

AI bắt đầu tính đến phản ứng của đối thủ. T02 và T03 (phòng thủ) được giải quyết tốt. Tuy nhiên các tình huống đòi hỏi kế hoạch 3+ bước (T06, T11) vẫn chưa được phát hiện.

+ Độ sâu 3 — Bắt đầu nhìn thấy chiến lược:

T7 vẫn có sai lệch do bẫy điểm số cục bộ — AI chọn nước có điểm cao ngay lập tức thay vì nước đi chiến lược hơn. Điều này cho thấy giới hạn của hàm đánh giá hiện tại, không chỉ giới hạn của độ sâu.

+ Độ sâu 4 — Rủi ro lệch hướng ở độ sâu lớn:

Ở T02 và T9 độ sâu 4, AI lại chọn sai. Nguyên nhân: khi nhìn sâu 4 bước, điểm số tích lũy từ nhiều cửa sổ heuristic dẫn đến một nước đi có tổng điểm cao hơn về mặt heuristic nhưng không tối ưu trong bối cảnh ngắn hạn. Đây là vấn đề cổ điển của 'horizon effect' — AI bị đánh lừa bởi các trạng thái tốt nằm ngay sau giới hạn tìm kiếm.

=> Kết luận: Độ sâu 2–3 cho kết quả tốt nhất trong thực tế với thời gian chấp nhận được (<300ms). Độ sâu 4 cải thiện một số tình huống nhưng gây lỗi ở một số kịch bản do tương tác phức tạp giữa heuristic và độ sâu tìm kiếm.

V. TỔNG KẾT

1. Ưu điểm và hạn chế của chương trình

1.1. Ưu điểm

- Hàm đánh giá trạng thái sử dụng kỹ thuật cửa sổ trượt kết hợp hệ thống trọng số ưu tiên phòng thủ cực cao (Phạt -80,000 khi đối thủ có chuỗi 3). Điều này giúp AI nhạy bén, phát hiện và dập tắt ngay lập tức các đợt tấn công 1-ply của người chơi ở mọi hướng (ngang, dọc, chéo).
- Alpha-Beta giảm 94.2% số trạng thái so với Minimax, cho phép tìm kiếm ở độ sâu cao hơn trong cùng thời gian.
- Tính đúng đắn của Alpha-Beta được xác nhận: 0 phân kỳ nước đi so với Minimax trên toàn bộ kiểm thử.

1.2. Hạn chế

- Mất cân bằng trọng số: Điểm chặn là -80,000 nhưng điểm thưởng tấn công (chuỗi 3 của AI) chỉ là +5,000. Điều này khiến AI chơi quá an toàn, đôi khi mãi mê chặn đối thủ mà bỏ lỡ đường thắng của chính mình.
- Lỗi đếm đôi và bù trừ: Ở các cửa sổ hỗn hợp chứa cả X và O, hệ thống không cộng dồn điểm chính xác mà trả về 0. Đồng thời, kỹ thuật cửa sổ 4 ô đôi khi đếm trùng lặp các mối đe dọa chồng lên nhau, làm sai lệch tổng điểm.
Hàm đánh giá không phân biệt chuỗi 'mở hai đầu' (.XXX.) và 'chặn một đầu' (OXXX.), khiến AI đánh giá mức độ nguy hiểm chưa chính xác.
- Thiếu điểm thưởng cho chuỗi 2 quân của máy (mine==2), AI không có động lực xây chuỗi từ sớm.
- Không nhận diện được chiến lược fork (tạo 2 mối đe dọa đồng thời), một kỹ thuật quan trọng trong cờ Caro.
- Horizon effect ở depth 4 gây lỗi trên một số kịch bản do hàm đánh giá chưa đủ chính xác để bù đắp cho tầm nhìn sâu

1.3. Hướng cải tiến

- Trường hợp chơi tốt: AI xử lý hoàn hảo trong các tình huống khai cuộc, trung cuộc đơn giản, hoặc khi có đe dọa rõ ràng trên 1 đường thẳng (Các kịch bản T01 đến T03, T05).
- Trường hợp chưa hợp lý:
 - + Bị "mù" trước chuỗi 3 mở hai đầu (.XXX.) do hàm đánh giá không phân biệt được chuỗi mở hai đầu và chuỗi bị chặn một đầu.
 - + Bỏ sót đường thắng của chính mình nếu chuỗi thắng đó nằm đè lên một cửa sổ có chứa 1 quân của đối phương
 - + AI không tìm ra chiến lược fork ở cả depth 3 và 4. Nguyên nhân là hàm đánh

giá không thưởng cho 'chuỗi đe dọa kép' — chỉ đánh giá từng cửa sổ 4 ô độc lập mà không nhận diện tổ hợp 2 chuỗi song song.

+ T09 depth 4 và T02 depth 4: Horizon effect khiến AI chọn nước đi sai khi nhìn quá sâu mà heuristic không đủ chính xác.

- Dựa trên thực trạng của chương trình, nếu tiếp tục phát triển, nhóm sẽ ưu tiên 3 cải tiến cốt lõi sau:

+ Cải tiến Hàm đánh giá: Tách biệt việc chấm điểm quân Máy và quân Người để tránh xung đột cửa sổ hỗn hợp. Bổ sung việc nhận diện trực tiếp chuỗi "3 mở hai đầu" (.XXX.) với mức phạt cực nặng (-120,000) thay vì chỉ dùng cửa sổ 4 ô. Thêm phân biệt chuỗi mở hai đầu (nhân hệ số 1.5–2x), thêm thưởng cho chuỗi 2 quân liên tiếp.

+ Thêm nhận diện fork pattern: đánh giá số lượng chuỗi đe dọa đồng thời, không chỉ đánh giá từng chuỗi độc lập.

+ Áp dụng move ordering: sắp xếp các nước đi theo điểm heuristic trước khi mở rộng, tăng hiệu quả cắt nhánh Alpha-Beta thêm 30–50%.

+ Iterative deepening: thay vì tìm cố định depth=N, tăng dần depth từ 1 đến khi hết thời gian cho phép, kết hợp với bộ nhớ lưu kết quả (transposition table)

+ Sắp xếp nước đi: Trước khi đưa danh sách nước đi vào vòng lặp Alpha-Beta, nhóm sẽ dùng hàm Heuristic để sắp xếp các nước đi tiềm năng nhất lên đầu. Điều này sẽ giúp Alpha-Beta cắt nhánh sớm hơn và nhiều hơn, đưa tỷ lệ tối ưu tiệm cận mức lý tưởng.

+ Tích hợp Bảng chuyển vị: Lưu trữ kết quả của các trạng thái bàn cờ đã xét vào bộ nhớ đệm (Cache) để tái sử dụng, tránh việc AI phải lặp lại phép tính cho những thế cờ trùng lặp. Cải tiến này hứa hẹn tăng tốc độ tính toán lên gấp 3 đến 5 lần.

2. Link repository Github của nhóm

https://github.com/manhngoc2682005-jpg/23021316_23021370

3. Tài liệu tham khảo và phần đã tham khảo từ các repo mẫu

- Tài liệu môn học: Slide bài giảng môn Trí tuệ Nhân tạo

- Sách tham khảo nền tảng: Artificial Intelligence: A Modern Approach (Stuart Russell & Peter Norvig) - Chương 5: Adversarial Search

- Tài liệu lập trình:

+ Thư viện tkinter: Xây dựng giao diện đồ họa.

+ Thư viện time, copy: Đo lường hiệu năng và xử lý ma trận trạng thái.

- Phần đã tham khảo từ các Repository / Tutorial mẫu:

+ Nguồn tham khảo: Repository Connect4-Python của lập trình viên / nhà nghiên cứu Keith Galli (Link: <https://github.com/KeithGalli/Connect4-Python>).

+ Phân kế thừa: Kế thừa ý tưởng cốt lõi về việc sử dụng Kỹ thuật Cửa sổ trượt (Sliding Window) có kích thước 4 ô để thiết kế hàm đánh giá (Heuristic Evaluation). Đồng thời, nhóm cũng áp dụng triết lý gán trọng số bất đối xứng (Asymmetric Weighting) từ mã nguồn này, trong đó đặt mức điểm phạt cực kỳ nặng khi đối phương hình thành chuỗi 3 quân nhằm kích hoạt cơ chế phòng thủ tức thời của AI.